IndusFE: A Real-World Benchmark for Component-Based Front-End Code Generation

Anonymous Author(s)
Submission Id: 5900

Abstract

Multimodal large language models (MLLMs) show promise for front-end code generation, yet existing benchmarks primarily evaluate visual fidelity, largely ignoring the functional requirements and coding standards that govern production environments. We propose **IndusFE**, a benchmark that frames industrial front-end development as generating component-based code from three complementary inputs: design mockups, product requirement documents (PRDs), and component library specifications. From real production webpages we abstract 87 business components and construct 1,028 samples via a multi-agent pipeline with human verification. Our evaluation framework jointly measures executability, structural correctness, requirement coverage, and design system compliance. Evaluating state-of-the-art models in a minimal execution-feedback agent scaffold shows that they achieve strong visual similarity but frequently hallucinate invalid component attributes; fine-tuning a 7B model on IndusFE closes this compliance gap, which validates the benchmark's utility.

CCS Concepts

• **Computing methodologies** → **Computer vision**; **Natural language generation**; *Machine learning*; • **Information systems** → *Multimedia information systems.*

Keywords

code generation, front-end development, multimodal large language models, benchmark, design system compliance

ACM Reference Format:
Anonymous Author(s). 2026. IndusFE: A Real-World Benchmark for Component-Based Front-End Code Generation. In *Proceedings of ACM Multimedia 2026 (ACM MM '26).* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Implementing front-end pages from design mockups [2] is a repetitive yet critical task in software engineering, and recent MLLMs [20, 23] have shown promise in automating it [32]. Existing benchmarks model this as an Image-to-Code mapping problem, overlooking the upstream constraints central to industrial practice. Design2Code [35], Interaction2Code [38], and DesignBench [39] have progressively broadened the scope from static layouts to interactive and multi-framework settings, yet none requires models to satisfy functional requirements or adhere to design system specifications.

ACM MM '26, Rio de Janeiro, Brazil
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

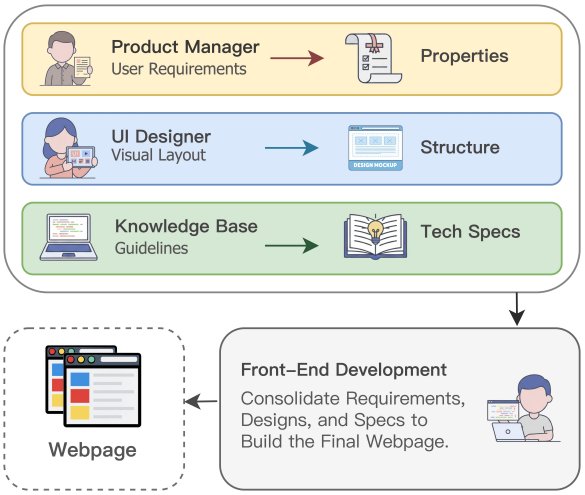

Figure 1: Three information sources in industrial front-end development. UI mockups define layout; PRDs specify functional logic and data bindings; component documentation constrains allowable APIs.

Design mockups specify layout but not business logic; functional semantics such as data binding and state transitions reside in PRDs, and component APIs and usage rules are defined in development documentation. As shown in Figure 2 [36], modern industrial teams predominantly adopt component-driven development (CDD), where engineers assemble predefined library components rather than writing raw HTML. This paradigm demands that code generators simultaneously satisfy visual, functional, and specification constraints, a requirement absent from all prior benchmarks.

We propose **IndusFE**, a benchmark that mirrors this industrial workflow. Models receive design mockups, PRDs, and component documentation as three complementary inputs and must produce component-based code conforming to strict design system rules (Table 1). We construct 1,028 samples from real production webpages via a multi-agent visual reverse-engineering pipeline, abstracting 87 business components with standardized attribute schemas. Both the visual inputs and the factual content of PRDs are grounded in real-world production data: design mockups are derived by masking fields from actual rendered pages, and PRD fields are extracted via static compilation rather than model inference, preserving realistic data distributions. A multidimensional evaluation framework assesses executability, structural fidelity, requirement coverage, and specification compliance, beyond visual similarity alone. Evaluating models in a minimal agent scaffold with browser-rendering feedback, we find

Table 1: Comparison of IndusFE with existing benchmarks. Spec.: Compliance with Design System constraints.

Benchmark	Samples	Real	Framework	Task Spec.	Evaluation Metrics
WebSight [18]	2M	×	×	Image	Visual
WebCode2M [12]	2.56M	✓	×	Image	Visual
Design2Code [35]	484	✓	×	Image	Visual
Interaction2Code [38]	127	✓	×	Image	Visual, Interaction
DesignBench [39]	900	✓	✓	Image	Visual
IndusFE (Ours)	**1,028**	✓	✓	**Image, PRD, Doc**	**Visual, Functional, Spec.**

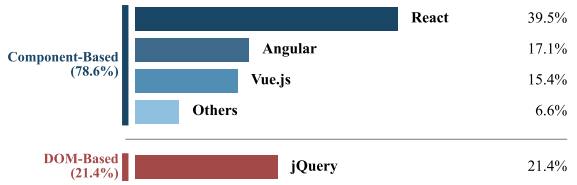

Figure 2: Front-end framework usage among developers. Component-based frameworks account for 78.6% of usage.

that general-purpose models frequently hallucinate invalid component attributes despite strong visual reasoning, while fine-tuning a 7B model on IndusFE closes this compliance gap.

Our contributions are:

- A formulation of industrial front-end generation as multimodal constraint satisfaction over visual, functional, and specification inputs.
- A dataset of 1,028 samples grounded in 87 real-world business components, built through a multi-agent pipeline with human verification.
- An evaluation suite covering execution correctness (CSR), structural similarity (TED), component recall, and design system compliance (Spec.), complementing visual metrics.
- Full release of the dataset, component library, pipeline code, and evaluation tools.[1]

2 Related Work

2.1 Front-End Code Generation

Front-end code generation dates back to Pix2Code [4], which first translated UI screenshots into code. WebSight [18] scaled this to millions of synthetic UI–HTML pairs for supervised fine-tuning, though synthetic data often diverges from real-world distributions. Design2Code [35] addressed this with 484 human-curated real-website samples, and WebCode2M [12] expanded to 2.56M samples while retaining real-world properties.

These benchmarks uniformly frame front-end development as vision-driven code generation, focusing on visual fidelity while ignoring upstream requirements. Interaction2Code [38] extended evaluation to dynamic behaviors (state transitions, user interactions), and DesignBench [39] introduced multi-framework evaluation with edit and repair tasks. All the above benchmarks generate purely presentational code such as static layouts or animations, without real backend data bindings or explicit requirement documents, and remain disconnected from industrial front-end workflows.

2.2 Code Generation with LLMs

LLM-based code generation has been extensively benchmarked on general programming tasks. Codex [5] introduced the HumanEval benchmark for Python function synthesis, and MBPP [1] broadened coverage to crowd-sourced programming problems. SWE-bench [17] raised the bar to repository-level issue resolution. SWE-agent [40] showed that a simple agent-computer interface lets models resolve such issues autonomously, and its minimal variant mini-SWE-agent [22] reduces the scaffold to a plain shell-action loop; we adopt this minimal design for evaluation (§4.2). On the model side, open-source code LLMs such as Code Llama [33], DeepSeek-Coder [13], StarCoder 2 [24], and WizardCoder [25] have narrowed the gap with proprietary systems. These benchmarks and models focus on algorithmic correctness and do not address the multimodal setting of front-end development.

2.3 Multimodal Understanding for UI

The Rico dataset [7] pioneered large-scale mobile UI data for design mining. GUI-oriented MLLMs such as CogAgent [15] have since demonstrated strong screen understanding, and web agent benchmarks such as WebArena [43] and Mind2Web [8] evaluate autonomous interaction with real websites. These efforts address UI understanding and interaction but not code generation under design system constraints.

3 The IndusFE Benchmark

3.1 Task Formulation

Traditional front-end development relies on hand-written HTML, CSS, and JavaScript, leading to fragmented code and limited reuse [10]. Modern industrial teams have largely replaced this approach with CDD, where engineers compose UIs from predefined, schema-constrained library components under strict Design Systems [31]. As illustrated in Figure 4, this shift replaces hardcoded markup with data-bound, reusable abstractions.

For code generation models, CDD shifts the core challenge from producing syntactically valid HTML to performing

[1] https://anonymous.4open.science/r/IndusFE-3F78

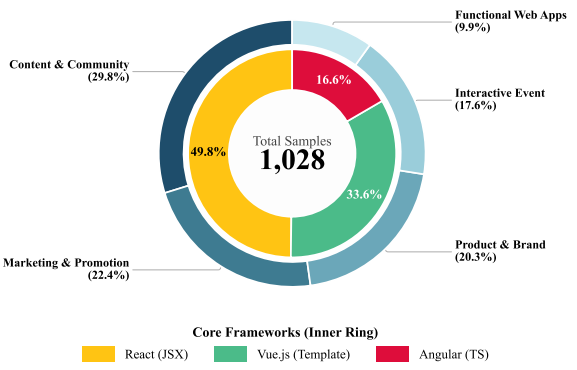

Figure 3: Composition of the IndusFE dataset. Outer ring: webpage category distribution across 5 content domains. Inner ring: target framework distribution.

semantic component composition: selecting the right components, configuring attributes within schema constraints, and establishing correct data bindings to backend services. A defining property of CDD is that complex interaction logic, state management, and backend integration are encapsulated within components: writing `<FollowButton uid={user.id} />` is sufficient to trigger the full follow workflow, including database state mutation, optimistic UI update, and animation feedback, because these behaviors are encapsulated within the component. The model's task is therefore to select the correct component and bind the correct props; the complexity of the underlying behavior is abstracted away by the design system. For instance, hardcoding "128 Followers" instead of binding `uid` produces a visually identical but functionally dead button. As shown in Figure 1, this requires jointly reasoning over three information sources: UI mockups for layout, PRDs for functional logic, and development documentation for API constraints, making front-end generation a multimodal constraint satisfaction problem rather than a visual translation task.

We first formalize the task, then describe the four-stage construction pipeline: (1) webpage crawling and filtering, (2) HTML cleaning and component library construction, (3) agent-driven code generation with human verification, and (4) inverse input synthesis for PRDs, design mockups, and component documentation. Through this pipeline, we construct 1,028 high-quality samples from real production webpages, each containing a design mockup, PRD, component documentation, and human-verified ground-truth code.

3.2 Webpage Crawling and Filtering

We crawled 10.9k webpages from real-world industrial business scenarios across 5 content domains (Content & Community, Marketing & Promotion, Product & Brand, Interactive Event, and Functional Web Apps) and generated corresponding page screenshots for each sample. All crawling strictly followed `robots.txt` directives, and the resulting URLs were reviewed by two annotators to exclude pages with restrictive terms of service. Any personally identifiable information (PII) detected during processing was replaced with synthetic placeholders; the dataset is released under CC-BY-NC 4.0. We then applied a multi-stage funnel: removing pages with overly short content or highly repetitive structures (near-duplicate MinHash Jaccard > 0.85) reduced the pool to 6.2k; excluding static-only pages without interactive elements left 3.4k candidates; after agent-driven code generation (§3.5) and manual verification, 1,028 samples passed all quality checks, yielding a 9.4% retention rate from the initial crawl.

As shown in Figure 3, the final dataset covers a diverse set of real business scenarios, ensuring broad coverage in both business forms and interaction complexity.

3.3 HTML Cleaning and Desensitization

We clean the filtered HTML by removing comments, scripts, and tracking snippets, then merge inline CSS into unified `<style>` blocks and extract layout-relevant style information. Redundant wrapper `<div>`s that serve no semantic purpose are collapsed to reduce DOM depth, and vendor-prefixed CSS rules are normalized to their standard forms. For privacy, all personally identifiable information detected via regex and NER is replaced with realistic synthetic placeholders, preserving the logical structure and data-binding patterns of the original page while ensuring no real user data is retained.

3.4 Component Library Construction

To construct the component library, we first count the occurrence frequency of each UI pattern across all 10.9k crawled pages and rank them by prevalence. We retain the highest-frequency patterns and merge near-duplicate variants, yielding 87 business components that collectively cover over 90% of the crawled samples, spanning five functional categories, e.g., navigation (`NavBar`), data display (`Leaderboard`), user interaction (`LotteryWheel`), media (`Carousel`), and form input (`SearchBar`).

For each component we define a strict attribute schema following three design principles: (1) minimal surface: only expose semantically necessary props, avoiding CSS-level styling knobs; (2) typed constraints: every prop has an explicit type and enumerated values where applicable, so compliance can be verified programmatically; and (3) data-binding slots: dynamic fields are first-class props rather than hardcoded text, ensuring generated code can connect to real backend services.

3.5 Agent-Driven Code Generation

We map raw webpages to component code using a collaborative multi-agent system [29, 37], with OpenAI o3 [27] as the backbone model. Three specialized agents communicate through structured artifacts in a sequential pipeline:

Stage I: Perception and Planning. An Observer Agent jointly analyzes screenshots and cleaned HTML, aligning visual regions with structural semantics. It outputs a structured component placement plan: an ordered list of UI regions, each annotated with the target component type, expected props,

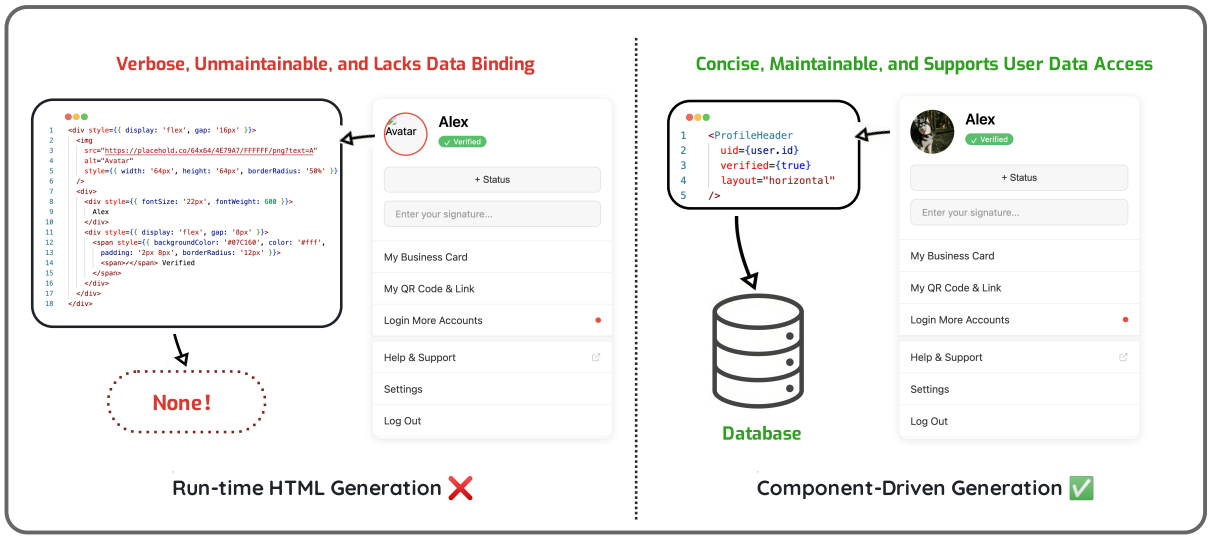

Figure 4: Hardcoded HTML (left) vs. component-based generation (right). The hardcoded approach produces static markup with no data connectivity; the component-based approach uses semantic properties for data binding, enabling reuse and backend integration.

and spatial layout constraints. This plan serves as the shared specification consumed by downstream agents.

Stage II: Collaborative Execution. A Code Generation Agent receives the placement plan along with the original screenshots and HTML, and builds a layout-preserving code skeleton that mirrors the spatial structure. A Component Substitution Agent then takes the skeleton and the plan, replaces skeleton nodes with library component instances, and populates attribute values according to the planned props and data-binding fields.

Stage III: Execution-Based Review. A Reviewer Agent [26] renders the Stage II output in a headless Chromium browser with the full component library loaded. It collects runtime diagnostics (invalid props, missing bindings, layout collapse) and produces a structured error report. If issues are detected, the report is fed back to the Stage II agents for targeted correction, repeating until all checks pass or a maximum of 3 rounds is reached [11, 34].

All agent-generated outputs undergo manual verification and correction by two senior front-end engineers, who review visual correctness, attribute validity, and code style compliance, and fix any remaining issues.

3.6 Inverse Input Synthesis

Given the verified ground-truth code, we synthesize the three model inputs via inverse construction. We illustrate with a minimal example: suppose the ground truth contains `<Button uid={user.id} variant="primary"/>`.

Code → Schema → PRD. PRD construction proceeds in two strictly separated stages. *(i) Deterministic extraction.* A rule-based compiler parses each ground-truth file and emits a structured *fact table*: component names, prop–value bindings, data fields, layout hierarchy, and conditional logic branches. For the running example it yields the fact record `Button` with props `uid=user.id` and `variant="primary"`. This fact table is a direct, model-free derivative of the source code and constitutes the immutable semantic core of the PRD. *(ii) Semantic paraphrasing.* DeepSeek-V3 [6] converts each fact-table entry into natural language, replacing code tokens with human-readable descriptions (e.g., `#FFFFFF` → "pure white background"; `variant="primary"` → "a prominent action button"). The rewriting removes all prop names, component identifiers, and code syntax, leaving only functional intent. We then invite product managers to review the generated PRDs and filter residual implementation-specific wording; minor functional details are omitted in ∼15% of samples to introduce realistic incompleteness. The factual content—what components are needed, what data they bind, and what logic they implement—is fully determined by stage (i) and is invariant to the language model used in stage (ii).

Page → Mockup. The ground-truth code is modified by masking all compiler-extracted text fields, stripping inline and custom CSS styles (e.g., colors, fonts, backgrounds), and resetting components to their default visual state. Non-component markup styles are also removed. The modified code is then recompiled and re-rendered in a headless browser to produce the design mockup. The resulting screenshot preserves spatial layout and component placement but removes textual content, color schemes, and styling details, ensuring models must rely on PRDs and documentation rather than visual shortcuts.

PRD → Docs (RAG). Using RAG [19], we embed component descriptions with text-embedding-3-small and retrieve the top-k (k=5) schemas per PRD entry. For this sample, the retrieval returns `Button`, `IconButton`, `Link`, `Tag`, and `Card`.

Table 2: Fine-tuning configuration for Qwen2.5-VL-7B on IndusFE.

Hyperparameter	Value
Base Model	Qwen2.5-VL-7B-Instruct
Training Framework	LLaMA-Factory
Hardware	16× NVIDIA H20 (96 GB)
Precision	BF16 (full-precision LoRA)
LoRA Rank (r) / Alpha (α)	32 / 64
LoRA Dropout	0.05
Target Modules	All Linear Layers
Trainable Parameters	1.2%
Global Batch Size	16
Learning Rate	2×10^{-4} (Cosine, warmup 0.1)
Epochs	10
Max Context Length	16,384
Optimizer / Parallelism	AdamW / DeepSpeed ZeRO-2
Wall-Clock Time	∼6 hours

The same pipeline is used during evaluation for all models; retrieval recall on the test set is 93.7%.

By design, no single input suffices to reconstruct the ground truth: the PRD specifies the functional intent but not the target component or props; the mockup shows spatial layout but not the underlying data; the documentation lists candidate APIs but not which one applies. The mean ROUGE-L between PRD tokens and ground-truth code tokens is 0.08, indicating minimal lexical overlap.

3.7 Framework Diversity

Unlike single-framework benchmarks, IndusFE covers three major frameworks (Figure 3): React (49.8%), Vue.js (33.6%), and Angular (16.6%). The component library is defined in a framework-agnostic manner, and both TED and Spec. metrics operate on parsed ASTs, enabling unified scoring across frameworks.

4 Experiment Setup

We evaluate three proprietary models—Gemini-2.5-Pro, GPT-4o, and Claude 3.7 Sonnet[2]—alongside Qwen2.5-VL-7B-Instruct [3], fine-tuned on the IndusFE training set (828 samples). All models use temperature 1.0, top-p 0.95, and a maximum output length of 32K tokens per step, with a 256K-token budget over the full trajectory. Every model runs inside the same agent scaffold (§4.2), so measured differences reflect model capability rather than scaffold engineering. We fine-tune the 7B model with LoRA [16] using BF16 precision without quantization [9] to preserve multimodal fidelity. Table 2 lists the full configuration.

4.1 Evaluation Metrics

Following holistic evaluation principles [21], IndusFE evaluates along executability, structure, and compliance using six complementary metrics. CSR and Spec. are specific to this benchmark; the other four are standard metrics adapted to our setting. For the five quantitative metrics, the formal definitions are given in Eqs. (1)–(5); the MLLM-as-a-Judge score is described separately as a rubric-based auxiliary assessment.

- **Compilation Success Rate (CSR):** Fraction of generated code that compiles and renders in a headless Chromium browser with the component library loaded, free of syntax or undefined-component errors:

$$\text{CSR} = \frac{|\{i : \texttt{compile}(c_i) = \text{success}\}|}{N} \tag{1}$$

  where c_i is the i-th generated code sample and N is the total number of samples. This is a binary per-sample metric averaged globally.
- **Design System Compliance (Spec.):** Quantifies how well the generated code conforms to the component library schema $\mathcal{K}$. Each generated element $e \in E_{gen}$, covering component names, prop keys, and data-binding expressions, is checked for validity:

$$\text{Score}_{\text{Spec.}} = \frac{\sum_{e \in E_{gen}} \mathbb{I}(\texttt{valid}(e, \mathcal{K}))}{|E_{gen}|} \tag{2}$$

  where $\texttt{valid}(e, \mathcal{K})$ returns true if e is a recognized component, a schema-conforming prop, or a correctly bound dynamic field. Spec. measures precision over the generated element set. Component Recall penalizes missing components and TED penalizes structural omissions.
- **Tree Edit Distance (TED):** Normalized structural similarity based on the Zhang–Shasha algorithm [41] with the RTED optimization [28], computed between the generated component tree T_{gen} and the ground-truth tree T_{gt}:

$$\text{Score}_{\text{TED}} = 1 - \frac{\text{EditDist}(T_{gen}, T_{gt})}{|T_{gen}| + |T_{gt}|} \tag{3}$$

  where $\text{EditDist}(\cdot, \cdot)$ counts the minimum insert, delete, and relabel operations, and the denominator (sum of both tree sizes) represents the theoretical maximum edit cost. Higher is better, scaled to $[0, 1]$.
- **Component Recall:** Measures whether all PRD-specified components appear in the generated AST, computed via keyword matching and alias normalization (e.g., "lottery wheel" → `LotteryWheel`):

$$\text{Recall} = \frac{|C_{gen} \cap C_{prd}|}{|C_{prd}|} \tag{4}$$

  where C_{prd} is the set of components required by the PRD and C_{gen} is the set present in the generated code. This metric operates at the component level; prop-level and binding-level correctness are captured by Spec. Per-sample scores are macro-averaged across the test set.
- **CLIP Score:** Visual similarity between rendered screenshots of generated and ground-truth code using CLIP-ViT-B/32 [30]:

$$\text{CLIP} = \frac{\mathbf{v}_{gen} \cdot \mathbf{v}_{gt}}{\|\mathbf{v}_{gen}\| \, \|\mathbf{v}_{gt}\|} \tag{5}$$

  where $\mathbf{v}_{gen}$ and $\mathbf{v}_{gt}$ are the CLIP image embeddings of the rendered screenshots. We prefer CLIP over pixel-level

[2] gemini-2.5-pro-preview-06-05, gpt-4o-2024-11-20, claude-3-7-sonnet-20250219.

metrics such as FID [14] because CLIP captures semantic layout similarity rather than low-level texture differences.

- **MLLM-as-a-Judge (auxiliary):** Following LLM-as-a-Judge [42], o3 scores each sample on four dimensions (layout accuracy, component usage, visual fidelity, code quality) using a 1–10 scale. The model is prompted to produce a weighted overall score emphasizing layout and visual fidelity. To ensure reproducibility, we use deterministic decoding (temperature = 0, seed = 42). We compare the judge against annotations from three human evaluators on 200 samples (Pearson r=0.82, inter-annotator Fleiss' κ=0.74). We also score the same 200 samples with Claude Opus 4 as an alternative judge and obtain identical model rankings (Kendall's τ=1.0, sample-level Spearman ρ=0.91).

4.2 Agent Scaffold

We evaluate all models inside a minimal agent scaffold adapted from mini-SWE-agent [22, 40]. The scaffold maintains a linear message history: at each step the model emits a single shell action, the environment executes it, and the resulting observation is appended to the history. There are no specialized tools, no retrieval plugins, and no history compression, which keeps the comparison across models free of scaffold-specific confounds.

The initial task message carries the same three inputs for every sample: (1) the component knowledge base as TypeScript schema definitions, (2) the PRD content, and (3) the visual design image. It also states three constraints: use only library-defined components instead of raw HTML, apply dynamic data binding instead of hardcoding visible text, and control appearance via schema props instead of raw CSS. The target framework (React/Vue/Angular) is specified per sample. Within its workspace, the agent can write and edit code files, trigger a Playwright-based rendering check that compiles the current file and loads it in headless Chromium with the component library, and read the returned error log (syntax errors, undefined components, invalid props, blank-screen detection).

The agent iterates against this rendering feedback until the check passes or a budget of 30 steps is exhausted, then submits its code file; the submitted file is the sole artifact scored by all six metrics. In the modality-ablated settings, the corresponding input is removed from the task message while the scaffold is unchanged. This evaluation scaffold is separate from the multi-agent construction pipeline of §3.5, which is used only to build ground-truth data and never at evaluation time.

5 Results and Analysis

5.1 Main Results

The Full Input rows of Table 3 reveal a consistent compliance gap. General-purpose models achieve high visual fidelity, and the execution-feedback loop recovers most compilation errors, yet 8.9% of GPT-4o's submissions still fail to compile and

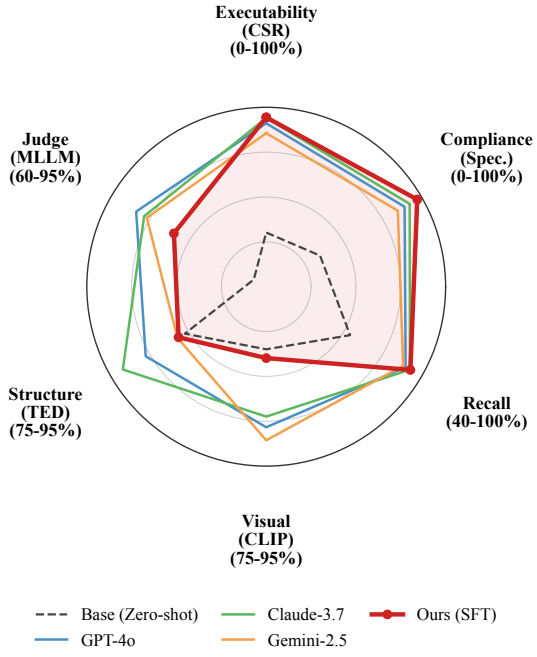

Figure 5: Radar chart comparing all evaluated models across six dimensions. The dashed line (Base) shows zero-shot Qwen2.5-VL-7B; the red line (Ours) shows performance after SFT. Axis ranges are adjusted per metric to highlight inter-model differences.

invalid component usage persists in submissions that render successfully. These models default to pre-trained knowledge of public libraries, frequently inventing plausible but non-existent props such as `shadow="true"` for a component that only accepts `variant="elevated"`. The 7B SFT model reaches 97.2% Spec. compliance, showing that internalizing the component schema via fine-tuning is more effective than in-context retrieval, even when models can iterate against rendering feedback.

Figure 5 visualizes this shift: the base Qwen2.5-VL-7B achieves reasonable visual and structural scores but near-zero compliance; after training on only 828 samples, Spec. and CSR both improve by over 0.6, showing that SFT embeds the schema into model parameters.

Our SFT model trails proprietary models on CLIP, TED, and MLLM despite leading on CSR and Spec., reflecting a capacity–compliance trade-off: fine-tuning the 7B model prioritizes schema compliance at the cost of layout fidelity, which larger models handle better through broader pre-training.

5.2 Ablation: Modality Contribution

The ablated rows in Table 3 confirm that each input modality resolves distinct ambiguities:

Table 3: Main results on IndusFE under Full Input and modality-ablated settings. Bold indicates the best; underline indicates the second best.

Model	Input Settings	CSR	TED	Recall	CLIP	MLLM	Spec.
Gemini-2.5	*w/o* Docs	0.3480	0.5721	0.7935	0.8422	7.0200	0.3624
	w/o Image	0.6730	0.6493	0.8461	0.5548	7.4100	0.7842
	w/o PRD	0.6920	0.7754	0.5163	0.8467	7.3300	0.8036
	Full Input	0.8560	0.8642	0.9288	**0.9212**	8.6900	0.8472
GPT-4o	*w/o* Docs	0.3860	0.6034	0.8147	0.8355	7.1400	0.3892
	w/o Image	0.7480	0.7052	0.8756	0.6259	7.6300	0.8317
	w/o PRD	0.7650	0.8214	0.5638	0.8663	7.5200	0.8409
	Full Input	0.9110	0.9051	0.9384	0.9067	**8.9300**	0.8907
Claude-3.7	*w/o* Docs	0.4920	0.6742	0.8542	0.8251	7.4500	0.4736
	w/o Image	0.7960	0.7811	0.8985	0.6452	7.9800	0.8664
	w/o PRD	0.8140	0.8493	0.5621	0.8570	7.8100	0.8791
	Full Input	0.9410	**0.9346**	0.9552	0.8948	8.7500	0.9236
Ours (Base)	**Full Input**	0.3020	0.8547	0.7241	0.8198	6.2800	0.3465
Ours (SFT)	*w/o* Docs	0.5840	0.7124	0.7638	0.7241	6.6800	0.6127
	w/o Image	0.8210	0.6624	0.9302	0.5867	6.4100	0.9021
	w/o PRD	0.8460	0.7962	0.5341	0.7883	6.6900	0.9104
	Full Input	**0.9440**	0.8628	**0.9568**	0.8296	8.0800	**0.9722**

Without Documentation. Spec. drops sharply for general models (GPT-4o: 0.89 → 0.39), as they rely on RAG for unseen standards. The SFT model retains 0.61, having encoded documentation into parametric memory during training.

Without Visuals. CLIP and TED decline sharply across all models. Models produce syntactically correct code but miss spatial layout. Recall remains stable: PRDs alone suffice for identifying which components to use, but visual input is needed to arrange them.

w/o PRDs. Component Recall drops sharply. Models cannot infer invisible business logic without explicit functional requirements. CSR and Spec. are less affected because visual cues and documentation still guide component selection, but the generated code lacks functional completeness.

5.3 Error Taxonomy

We independently labeled 100 randomly sampled compilation failures from GPT-4o's final submissions (Cohen's κ=0.81). Disagreements were resolved through discussion. Three dominant error types emerged:

- **Hallucination (45%):** The model generates attributes absent from the component schema. 72% of such props exist in popular external libraries rather than the target schema.
- **Topological Errors (30%):** Components are nested in structurally invalid ways, violating the atomic design hierarchy [10]. These errors often compile but produce broken layouts that fail Spec. checks.
- **Data Binding Failures (25%):** Dynamic data is replaced with hardcoded literals, breaking the data abstraction the component API requires.

Table 4: Per-framework CSR and Spec. (Full Input).

	React		Vue		Angular	
Model	CSR	Spec.	CSR	Spec.	CSR	Spec.
Gemini-2.5	0.884	0.868	0.856	0.842	0.772	0.795
GPT-4o	0.932	0.908	0.908	0.885	0.854	0.851
Claude-3.7	0.958	0.938	0.938	0.920	0.896	0.888
Ours (SFT)	**0.961**	**0.981**	**0.940**	**0.970**	**0.901**	**0.951**

Gemini and Claude show similar distributions (±5pp per category).

5.4 Per-Framework Analysis

Table 4 reports results broken down by target framework. React achieves the highest scores across all models, likely due to its larger share in pre-training corpora. Vue.js follows closely. Angular shows the largest gaps in CSR, where its template syntax causes more compilation failures. The SFT model maintains Spec. >0.95 across all three frameworks, indicating that fine-tuning transfers design system knowledge across syntactic variants.

5.5 Generalization to Unseen Components

To verify that the SFT model learns to follow documentation rather than memorizing the 72 seen training components, we hold out 15 components and retrain without any samples containing them. At inference time the model receives only the held-out component's documentation via RAG. We group the 15 components into three complexity tiers by prop count: Simple (≤5 props), Medium (6–10), and Hard (>10).

Table 5: Generalization to 15 unseen components by complexity.

Complexity	#Props	n	CSR	Spec.
Simple	≤5	38	0.943	0.951
Medium	6–10	41	0.904	0.917
Hard	>10	33	0.852	0.869
Avg. (unseen)		112	0.902	0.914
Seen components		88	0.963	0.972

Table 6: Synthetic (S) vs. real-world (R) PRDs (n=50).

	CSR		Recall		Spec.		CLIP	
Model	S	R	S	R	S	R	S	R
Gemini-2.5	.860	.840	.920	.900	.848	.842	.922	.916
GPT-4o	.920	.900	.940	.920	.892	.884	.906	.902
Claude-3.7	.940	.920	.950	.930	.924	.918	.894	.888
Ours	.945	.925	.960	.940	.972	.968	.830	.826
Mean $\|\Delta\|$	2.0pp		2.0pp		0.6pp		0.5pp	

As shown in Table 5, performance degrades gracefully with complexity. Simple components are handled nearly as well as seen ones, while Hard components with many props and callback handlers show a larger gap. The overall 5.8pp Spec. gap confirms that fine-tuning transfers documentation-following skills beyond the training set, though complex APIs remain challenging.

5.6 PRD Validity Study

A natural concern is whether synthesized PRDs yield the same evaluation conclusions as real-world ones. To test this, we collected 50 real PRDs from production projects, each paired with its corresponding webpage implemented by front-end engineers. We then applied our inverse input synthesis pipeline (§3.5) to these webpages to produce synthetic PRDs. We ran all four models under both PRD sources with identical mockups and documentation.

As Table 6 shows, model rankings are perfectly preserved (Kendall's τ=1.0; sample-level Spearman ρ=0.94), with a mean absolute difference of only 1.3pp. Real-world PRDs mainly reduce Recall (2pp), while schema-dependent metrics (CSR, Spec.) are nearly unaffected.

The close agreement confirms that our synthetic PRDs faithfully preserve the semantic content of real-world requirements. The inputs provided to models are clear and complete, not degraded or adversarial. IndusFE therefore evaluates a model's ability to understand natural language requirements and translate them into specification-compliant code, requiring long-context comprehension across PRDs, documentation, and mockups. It does not test robustness to noisy or erroneous inputs.

6 Limitations

Our benchmark has two main limitations: (1) the component library covers 87 components and does not exhaust all real-world scenarios; (2) while our PRD validity study (§5.6) shows consistent model rankings between synthetic and human-authored PRDs, the synthesized PRDs may still underrepresent edge cases found in rapidly evolving product requirements.

7 Conclusion

We introduced **IndusFE**, a benchmark that models industrial front-end development as a multimodal constraint satisfaction problem over visual designs, product requirements, and component specifications. Through a visual reverse-engineering pipeline, we constructed 1,028 high-quality samples and a multidimensional evaluation framework. Experiments show a clear compliance gap: even with execution feedback in an agent scaffold, generalist models produce visually accurate code but frequently hallucinate invalid component attributes, and fine-tuning a 7B model on IndusFE closes this gap in execution correctness and design system compliance. We release all data and code to support further research toward production-ready front-end code generation.

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021).

[2] Batuhan Aşıroğlu, Büşta Rümeysa Mete, Eyyüp Yıldız, Yağız Nalçakan, Alper Sezen, Mustafa Dağtekin, and Tolga Ensari. 2019. Automatic HTML Code Generation from Mock-Up Images Using Machine Learning Techniques. In *2019 Scientific Meeting on Electrical-Electronics & Biomedical Engineering and Computer Science (EBBT)*. 1–4. doi:10.1109/EBBT.2019.8741736

[3] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. 2025. Qwen2.5-VL Technical Report. arXiv:2502.13923 [cs.CV] doi:10.48550/arXiv.2502.13923

[4] Tony Beltramelli. 2018. pix2code: Generating Code from a Graphical User Interface Screenshot. In *Proceedings of the ACM SIGCHI Symposium on Engineering Interactive Computing Systems* (Paris, France) *(EICS '18)*. Association for Computing Machinery, New York, NY, USA, Article 3, 6 pages. doi:10.1145/3220134.3220135

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[6] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, et al. 2024. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437* (2024). doi:10.48550/arXiv.2412.19437

[7] Biplab Deka, Zifeng Huang, Chad Franzen, Joshua Hibschman, Daniel Afergan, Yang Li, Jeffrey Nichols, and Ranjitha Kumar. 2017. Rico: A Mobile App Dataset for Building Data-Driven Design Applications. In *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology*. 845–854.

[8] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a

Generalist Agent for the Web. In *Advances in Neural Information Processing Systems*, Vol. 36.

[9] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 36.

[10] Brad Frost. 2016. *Atomic Design*. Brad Frost Pittsburgh.

[11] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. https://openreview.net/forum?id=Sx038qxjek

[12] Yi Gui, Zhen Li, Yao Wan, Yemin Shi, Hongyu Zhang, Bohua Chen, Yi Su, Dongping Chen, Siyuan Wu, Xing Zhou, Wenbin Jiang, Hai Jin, and Xiangliang Zhang. 2025. WebCode2M: A Real-World Dataset for Code Generation from Webpage Designs. In *Proceedings of the ACM on Web Conference 2025* (Sydney NSW, Australia) *(WWW '25)*. Association for Computing Machinery, New York, NY, USA, 1834–1845. doi:10.1145/3696410.3714889

[13] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y.K. Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).

[14] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. 2017. GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium. In *Advances in Neural Information Processing Systems*, Vol. 30.

[15] Wenyi Hong, Weihan Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, and Jie Tang. 2024. CogAgent: A Visual Language Model for GUI Agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 14281–14290.

[16] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net. https://openreview.net/forum?id=nZeVKeeFYf9

[17] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations*.

[18] Hugo Laurençon, Léo Tronchon, and Victor Sanh. 2024. Unlocking the conversion of Web Screenshots into HTML Code with the WebSight Dataset. arXiv:2403.09029 [cs.HC] https://arxiv.org/abs/2403.09029

[19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. 9459–9474.

[20] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. In *International Conference on Machine Learning*. PMLR, 19730–19742.

[21] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. 2023. Holistic Evaluation of Language Models. *Transactions on Machine Learning Research* (2023).

[22] Kilian Lieret, John Yang, Carlos E. Jimenez, and Ofir Press. 2025. mini-SWE-agent: The 100 line AI agent that solves GitHub issues. https://github.com/SWE-agent/mini-swe-agent. GitHub repository.

[23] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual Instruction Tuning. In *Advances in Neural Information Processing Systems*, Vol. 36.

[24] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. 2024. StarCoder 2 and The Stack v2: The Next Generation. *arXiv preprint arXiv:2402.19173* (2024).

[25] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2024. WizardCoder: Empowering Code Large Language Models with Evol-Instruct. In *International Conference on Learning Representations*.

[26] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10-16, 2023*. http://papers.nips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html

[27] OpenAI. 2025. Introducing OpenAI o3 and o4-mini. https://openai.com/index/introducing-o3-and-o4-mini/

[28] Mateusz Pawlik and Nikolaus Augsten. 2011. RTED: A Robust Algorithm for the Tree Edit Distance. *Proceedings of the VLDB Endowment* 5, 4 (2011), 334–345. doi:10.14778/2095686.2095692

[29] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 15174–15186. doi:10.18653/v1/2024.acl-long.810

[30] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *International Conference on Machine Learning*. PMLR, 8748–8763.

[31] Mythili Raju. 2026. 32 Best Web Development Frameworks For 2026. https://www.testmuai.com/blog/best-web-development-frameworks/ Formerly LambdaTest. Accessed: 2026-04-02.

[32] Alex Robinson. 2019. Sketch2code: Generating a website from a paper mockup. arXiv:1905.13750 [cs.CV] https://arxiv.org/abs/1905.13750

[33] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* (2023).

[34] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36.

[35] Chenglei Si, Yanzhe Zhang, Ryan Li, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. 2025. Design2Code: Benchmarking Multimodal Code Generation for Automated Front-End Engineering. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 3956–3974. doi:10.18653/v1/2025.naacl-long.199

[36] Stack Overflow. 2024. *Stack Overflow Developer Survey 2024*. https://survey.stackoverflow.co/2024/ N=65,437 respondents.

[37] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *First Conference on Language Modeling (COLM)*. https://openreview.net/forum?id=BAakY1hNKS

[38] Jingyu Xiao, Yuxuan Wan, Yintong Huo, Zixin Wang, Xinyi Xu, Wenxuan Wang, Zhiyao Xu, Yuhang Wang, and Michael R. Lyu. 2025. Interaction2Code: Benchmarking MLLM-based Interactive Webpage Code Generation from Interactive Prototyping. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 241–253. doi:10.1109/ASE63991.2025.00028

[39] Jingyu Xiao, Ming Wang, Man Ho Lam, Yuxuan Wan, Junliang Liu, Yintong Huo, and Michael R. Lyu. 2025. DesignBench: A Comprehensive Benchmark for MLLM-based Front-end Code Generation. arXiv:2506.06251 [cs.SE] https://arxiv.org/abs/2506.06251

[40] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37.

[41] Kaizhong Zhang and Dennis Shasha. 1989. Simple Fast Algorithms for the Editing Distance between Trees and Related Problems. *SIAM J. Comput.* 18, 6 (1989), 1245–1262.

[42] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, Vol. 36.

[43] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Graham Neubig, and Jamie Callan. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations*.